

OutlierDetect Pipeline and Module Guide

Generated from the current repository state

June 18, 2026

Contents

1	Scope	3
2	End-to-End Pipeline	3
2.1	Train/Test Semantics	4
3	Shared Data Contracts	4
3.1	ProfileInput	4
3.2	NormalizationStats	4
3.3	NuisanceBias and Result	5
4	Input and Ingestion Modules	5
4.1	netcdf_backend.py	5
4.2	argo.py	5
4.3	en4.py	6
4.4	parquet.py	6
5	Physics, Corrections, and Feature Construction	6
5.1	density.py	6
5.2	corrections.py	7
5.3	features.py	7
6	Inference Path	8
6.1	tool.py	8
6.2	Heuristic baseline	8

6.3	Neural wrapper	9
7	Synthetic Training Data	9
7.1	Shared synthetic builder	9
7.2	training/synthetic.py	9
7.3	training/argo.py and training/en4.py	10
8	Training Dataset and Batching	10
8.1	training/dataset.py	10
9	Model Architecture	10
9.1	model.py	10
10	Training Objective and Loop	11
10.1	training/train.py	11
10.2	Checkpointing	12
11	Runtime Configuration and CLI	12
11.1	runtime_config.py	12
11.2	cli.py	12
12	Artifacts and Run Outputs	13
12.1	training/artifacts.py	13
13	Module Map	13
14	Closing Notes	14

1 Scope

OutlierDetect is a profile-level quality-control and reconstruction system for sparse CTD-SRDL-style ocean profiles. The package is organized around one shared data contract, one feature builder, one physical correction layer, and two execution paths: an inference path that returns a structured result object, and a training path that creates synthetic labels from clean source profiles.

This report describes the current pipeline in the repository as it exists now, including the module boundaries and the way the CLI, config layer, physics layer, feature builder, synthetic training generator, model, and artifact writer fit together.

2 End-to-End Pipeline

At a high level, the system moves through these stages:

1. Read a profile source from NetCDF or parquet.
2. Convert it into a `ProfileInput` object with pressure, temperature, salinity, optional residuals, uncertainty scales, and metadata.
3. Build per-level physical and statistical features, including density diagnostics and nuisance-correction features.
4. Run either the heuristic baseline or the neural model to obtain profile-level and point-level outlier probabilities.
5. Optionally reconstruct a stable temperature/salinity profile on a fixed pressure grid.
6. Persist the result as a structured `Result` object, JSON sidecars, plots, checkpoints, and run metadata.

Training uses the same input contract, but adds one extra stage before feature construction: a clean source profile is degraded synthetically to create sparse, noisy, labeled training examples. The training path is therefore:

1. Read clean Argo or EN4 source profiles.
2. Generate sparse synthetic examples with known labels.
3. Compute normalization statistics from the training side only.
4. Pad variable-length profiles into batches.
5. Train the multitask transformer with profile QC, point QC, nuisance regression, and reconstruction losses.
6. Save progress JSON, plots, and a checkpoint.

2.1 Train/Test Semantics

The current training CLI distinguishes between the source used for model fitting and the held-out source used for evaluation:

- `-train-root` is the canonical training input root.
- `-test-root`, when provided, is treated as a separate held-out root.
- The training side uses the adjusted/QC-filtered reader path.
- The held-out side uses the raw reader path, meaning TEMP/PSAL without QC masking for Argo and the equivalent raw selection for EN4.
- If `-test-root` is omitted, the training root is split internally with `-val-fraction`.

That split matters because it lets you compare performance on adjusted/corrected training sources against a raw held-out distribution later.

3 Shared Data Contracts

The package keeps data exchange simple on purpose. Most modules pass around a small set of dataclasses rather than large opaque structures.

3.1 ProfileInput

`ProfileInput` in `src/outlierdetect/data.py` is the central input object. It holds:

- pressure, temperature, and salinity arrays;
- optional residuals against a reference profile;
- uncertainty scales for temperature, salinity, vertical heave, and heave-derived uncertainty;
- an optional T/S correlation coefficient;
- day-of-year, profile ID, and arbitrary metadata.

It validates that pressure is one-dimensional, finite, non-negative, and sorted, and that temperature and salinity each contain at least one finite value.

3.2 NormalizationStats

`NormalizationStats` stores per-run mean and standard deviation values for temperature and salinity. Training computes it from the training examples only, then reuses it when normalizing inputs and reconstruction targets.

3.3 NuisanceBias and Result

`NuisanceBias` is the backwards-compatible summary for the local linear nuisance terms:

$$[a_t, b_t, a_s, b_s]$$

where the slopes are expressed per km in the correction layer and per normalized pressure in the feature builder.

`Result` is the main output object returned by predictors. It contains:

- `profile_bad_probability`
- point-level probabilities for temperature, salinity, and density inconsistency
- `nuisance_bias`
- `correction_posterior`
- reconstructed pressure, temperature, salinity, and uncertainty bands
- feature names and diagnostics
- the originating profile ID

4 Input and Ingestion Modules

4.1 netcdf_backend.py

`src/outlierdetect/netcdf_backend.py` owns the low-level NetCDF opening and array extraction logic. It tries multiple backends so the rest of the code does not care which reader library is available. The reader path is designed to survive the common variations you see in oceanographic files:

- `netCDF4`
- `h5py`
- `scipy.io.netcdf_file`
- a padded fallback for truncated HDF5-style files

Higher-level modules use the helper functions from here to read variables, QC flags, and profile metadata without repeating backend-specific code.

4.2 argo.py

`src/outlierdetect/argo.py` parses Argo float files into `ArgoProfile` objects. It is responsible for:

- discovering profile structure in single-profile and multi-profile files;
- reading the standard Argo variables;
- applying QC masking when requested;
- extracting metadata such as cycle number, float WMO, JULD, latitude, and longitude;
- converting the result into `ProfileInput`.

The reader has two important modes:

- default mode uses `TEMP\_ADJUSTED`, `PSAL\_ADJUSTED`, and Argo QC flags;
- raw mode uses `TEMP` and `PSAL` directly and skips QC masking.

That raw mode is what the held-out test root now uses.

4.3 `en4.py`

`src/outlierdetect/en4.py` does the same job for EN4 monthly files, but with EN4-specific variable discovery. It selects pressure, temperature, and salinity from the candidate names exposed by the dataset and can also operate in raw mode for held-out evaluation.

The key point is that EN4 is normalized into the same `ArgoProfile` shape, which keeps the downstream synthetic builder source-agnostic.

4.4 `parquet.py`

`src/outlierdetect/parquet.py` provides raw data export and iteration over parquet-backed profile data. This is useful when you want to stage NetCDF data once and then iterate over a lighter-weight representation later. It sits outside the core physical pipeline, but it is part of the ingest path.

5 Physics, Corrections, and Feature Construction

5.1 `density.py`

`src/outlierdetect/density.py` replaces the old linear density proxy with TEOS-10/GSW-based `sigma0` calculations. The module provides:

- `density\_proxy` as a compatibility alias for `sigma0`;
- `sigma0\_from\_ts` for T-S plots;
- `inversion\_metrics` for density inversions and inversion magnitude;

- `stable\_project\_salinity\_only` for salinity-only stability projection.

The density diagnostics matter in two places:

- the feature builder uses them to flag unstable structure;
- the heuristic baseline uses them directly when scoring point anomalies and reconstructions.

5.2 `corrections.py`

`src/outlierdetect/corrections.py` implements a Gaussian nuisance correction model over the 4D latent vector

$$c = [a_t, b_t, a_s, b_s]^T.$$

The model is deliberately physical:

- temperature and salinity offsets are allowed to differ;
- slopes are softly constrained;
- temperature and salinity terms may be correlated;
- the posterior can be refined with point weights from the QC heads.

`CorrectionPrior` stores the mean and covariance of the latent correction terms. `CorrectionPosterior` stores the updated Gaussian posterior, the debiased residuals, the pointwise correction deltas, prior tension, information gain, and diagnostic status.

The prior is used by both the heuristic and the neural feature path. The posterior is not a final tag-adjustment system; it is an explicit nuisance layer for bias-insensitive QC.

5.3 `features.py`

`src/outlierdetect/features.py` is the central feature entry point for both inference and training. It builds one feature row per observed level and returns a `FeatureBatch` containing:

- `level\_features`
- `feature\_names`
- a valid-level mask
- diagnostics such as detrend fits, correction posterior, and feature version

The current feature vector has 46 columns. It covers:

- pressure geometry: normalized pressure, neighbor gaps, edge flags;
- raw state: temperature and salinity, optionally normalized by training stats;

- local shape: first and second derivatives of T and S;
- density: sigma0 proxy and inversion magnitude;
- residuals: residual T/S, z-scores, detrended residuals, and detrended z-scores;
- correction features: posterior deltas, debiased residuals, correction tension, information gain, and constraint strengths;
- uncertainty inputs: `sigma\_t`, `sigma\_s`, `sigma\_vert`, `sigma\_heave\_t`, `sigma\_heave\_s`;
- contextual inputs: `rho\_ts`, day-of-year sine/cosine, and presence flags.

The builder also computes robust or standard linear detrends of residuals against pressure, which gives the model a cheap way to reason about coherent biases without reconstructing the full reference profile.

6 Inference Path

6.1 `tool.py`

`src/outlierdetect/tool.py` provides the two predictor-facing classes:

- `Heuristic`, a transparent baseline;
- `Neural`, the trainable wrapper around the transformer model.

Both predictors return the same `Result` object, so downstream consumers do not need to care whether the scores came from a rule-based path or a learned path.

6.2 Heuristic baseline

The heuristic path is intentionally explicit. It:

1. builds level features;
2. computes point-level bad probabilities from detrended z-scores, curvature, and density inversion magnitude;
3. optionally refits the correction posterior with point weights;
4. combines point scores into a profile-level bad probability;
5. reconstructs temperature and salinity on a pressure grid when enabled.

The scoring logic is simple enough to inspect and use as a debugging baseline. It is not the final system, but it anchors the API and makes regression testing straightforward.

6.3 Neural wrapper

The neural wrapper converts the same `ProfileInput` into features, passes them through the transformer, and maps the outputs back into the shared `Result` contract. It exists to keep the model interface stable while the learned system evolves.

7 Synthetic Training Data

7.1 Shared synthetic builder

`src/outlierdetect/training/builders.py` is the bridge from clean source profiles to synthetic training examples. Its job is to:

- preserve each profile’s native pressure sampling;
- sample sparse pressure indices per profile;
- feed the high-resolution source to the synthetic degradation function;
- carry metadata into the synthetic example.

There is no global resampling step that forces every source profile onto the same observed pressure grid before corruption.

7.2 `training/synthetic.py`

`training/synthetic.py` contains the actual corruption model. `degrade\_highres\_profile` takes a clean, high-resolution profile and turns it into a sparse labeled training example with:

- coherent T/S offsets and slopes;
- isolated spikes;
- pressure errors;
- occasional global bad profiles;
- heave/reference noise;
- density-inversion labels;
- a reconstruction truth grid.

This module is where the supervised learning problem is manufactured. The labels are synthetic by construction, which is why the rest of the training stack can be compact.

7.3 training/argo.py and training/en4.py

These modules adapt the source readers to the synthetic builder:

- `src/outlierdetect/training/argo.py` converts Argo profiles into synthetic examples;
- `src/outlierdetect/training/en4.py` does the same for EN4;
- both accept `use\_raw\_values`;
- both forward clean profiles into the shared corruption pipeline.

In the current pipeline, the training root uses adjusted/QC-filtered source data, while the held-out test root uses raw values. That is the mechanism that lets you compare performance against a different data preparation regime later.

8 Training Dataset and Batching

8.1 training/dataset.py

`src/outlierdetect/training/dataset.py` defines the dataset objects and the padding logic.

`ProfileExample` pairs a `ProfileInput` with labels. `ProfileLabels` stores:

- profile-level bad/good labels;
- point-level labels for temperature, salinity, and density inconsistency;
- nuisance means;
- reconstruction truth on a fixed pressure grid;
- optional extra metadata.

`ProfileDataset` lazily builds features when indexed. This keeps the representation simple and makes feature revisions cheap during development. `collate\_profiles` pads a batch of variable-length profiles into tensors and masks. `compute\_normalization\_stats` derives temperature and salinity scaling from the training examples only.

9 Model Architecture

9.1 model.py

`src/outlierdetect/model.py` defines the neural network used by the training loop.

`NetConfig` describes the model size:

- input dimension;
- hidden size;
- number of transformer layers and heads;
- dropout;
- grid size for reconstruction.

`Net` is a compact transformer encoder for irregular profiles. The input tensor has shape

$$[\text{batch}, \text{n_levels}, \text{input_dim}]$$

and a boolean mask marks valid levels.

The model has four heads:

- a point head for per-level QC logits;
- a profile head for profile-level QC;
- a nuisance head for the 4D correction posterior;
- a reconstruction head for temperature and salinity on a fixed grid.

The reconstruction branch combines a pooled profile embedding with encoded pressure-grid features, which lets the model predict a profile on a caller-defined standard grid without forcing the input observations onto that grid.

10 Training Objective and Loop

10.1 `training/train.py`

`src/outlierdetect/training/train.py` contains the loss definition and the per-epoch loop.

The total loss is a weighted sum of:

$$\mathcal{L} = \lambda_p \mathcal{L}_{profile} + \lambda_q \mathcal{L}_{point} + \lambda_n \mathcal{L}_{nuisance} + \lambda_{kl} \text{KL}(q||p) + \lambda_r \mathcal{L}_{recon} + \lambda_u \mathcal{L}_{uncertainty}.$$

Concretely:

- profile QC and point QC use binary cross-entropy with logits;
- nuisance regression uses a Gaussian negative log-likelihood;
- the nuisance head is regularized toward the physical prior;
- reconstruction uses a Gaussian NLL when reconstruction targets are present;
- the uncertainty term discourages pathological predicted uncertainty.

`fit\_model` runs alternating train and eval epochs and can prefix evaluation metrics with `val\_` or `test\_` depending on the configured evaluation label. That prefixing is what lets the same training loop report either split without changing downstream consumers.

10.2 Checkpointing

`save\_checkpoint` stores the model weights and a small metadata block. The metadata typically includes input dimension, grid size, feature names, normalization stats, and the resolved run configuration.

11 Runtime Configuration and CLI

11.1 `runtime_config.py`

`src/outlierdetect/runtime_config.py` owns typed configuration loading and path resolution. The config schema is split into dataclasses:

- `PathConfig`
- `HeaveConfig`
- `InputConfig`
- `TrainConfig`
- `PredictConfig`
- `AppConfig`

The loader reads TOML, resolves paths relative to the config file, and keeps backward compatibility with older `argo\_root` training configs by mapping them to `train\_root`.

11.2 `cli.py`

`src/outlierdetect/cli.py` is intentionally thin. It parses user-facing arguments, resolves the runtime config, and delegates to the lower-level modules.

The installed console entry points are:

- `outlierdetect`
- `outlier-detect`
- `outlierdetect-predict`
- `outlierdetect-train`

The top-level CLI also exposes config management:

- `outlierdetect config init`
- `outlierdetect config show`

- `outlierdetect config validate`

Other workflow flags include raw-to-parquet export, prediction, training, optional heave source selection, and input toggles for residuals, uncertainty scales, and seasonal phase. Prediction still uses `-argo-root`, because that flag names the inference input root. Training now uses `-train-root`.

12 Artifacts and Run Outputs

12.1 `training/artifacts.py`

`src/outlierdetect/training/artifacts.py` writes the per-epoch run artifacts. A training run typically produces:

- a run directory under the configured run root;
- a progress JSON file with epoch histories and selected examples;
- sampled reconstruction plots under `plots/epoch\_{XXX}/`;
- prediction JSON sidecars for the sampled profiles;
- a final checkpoint if an output path is configured.

The progress file records the current evaluation label, counts for train and eval examples, selected profile IDs, and the latest metrics snapshot. This makes it easy to inspect a run without loading the checkpoint.

13 Module Map

Module	Role	Key objects / functions
<code>src/outlierdetect/data.py</code>	Shared input/output dataclasses	<code>ProfileInput</code> , <code>Result</code> , <code>NuisanceBias</code> , <code>NormalizationStats</code>
<code>src/outlierdetect/density.py</code>	TEOS-10 density and stability	<code>density_proxy</code> , <code>inversion_metrics</code> , <code>stable_project_salinity_only</code>
<code>src/outlierdetect/corrections.py</code>	Bayesian nuisance correction	<code>CorrectionPrior</code> , <code>CorrectionPosterior</code> , <code>estimate_correction_posterior</code>
<code>src/outlierdetect/features.py</code>	Feature construction	<code>build_level_features</code> , <code>FeatureBatch</code> , <code>linear_detrend</code>
<code>src/outlierdetect/netcdf_backend.py</code>	Low-level file opening	backend selection helpers for NetCDF/HDF5 access
<code>src/outlierdetect/argo.py</code>	Argo profile reading	<code>ArgoProfile</code> , <code>read_argo_file</code> , <code>iter_argo_files</code>
<code>src/outlierdetect/en4.py</code>	EN4 profile reading	<code>read_en4_file</code> , <code>iter_en4_files</code>
<code>src/outlierdetect/parquet.py</code>	Parquet staging/export	<code>write_argo_parquet</code> , <code>iter_argo_parquet_profiles</code>

Module	Role	Key objects / functions
<code>src/outlierdetect/tool.py</code>	Inference wrappers	<code>Heuristic</code> , <code>Neural</code> , <code>Config</code>
<code>src/outlierdetect/model.py</code>	Transformer network	<code>NetConfig</code> , <code>Net</code> , <code>masked_mean</code>
<code>src/outlierdetect/runtime_config.py</code>	TOML and path resolution	<code>AppConfig</code> , <code>TrainConfig</code> , <code>PredictConfig</code> , loaders
<code>src/outlierdetect/cli.py</code>	Command-line entry points	<code>main</code> , <code>train_main</code> , <code>predict_main</code> , config commands
<code>src/outlierdetect/training/dataset.py</code>	Dataset and collation	<code>ProfileExample</code> , <code>ProfileLabels</code> , <code>ProfileDataset</code> , <code>collate_profiles</code>
<code>src/outlierdetect/training/synthetic.py</code>	Synthetic corruption	<code>degrade_highres_profile</code>
<code>src/outlierdetect/training/builders.py</code>	Shared synthetic builder	convert clean profiles into synthetic examples
<code>src/outlierdetect/training/argo.py</code>	Argo training bridge	<code>build_argo_synthetic_examples</code>
<code>src/outlierdetect/training/en4.py</code>	EN4 training bridge	<code>build_en4_synthetic_examples</code>
<code>src/outlierdetect/training/train.py</code>	Loss and epoch loop	<code>compute_loss</code> , <code>train_epoch</code> , <code>eval_epoch</code> , <code>fit_model</code>
<code>src/outlierdetect/training/artifacts.py</code>	Run artifacts	<code>TrainingRunWriter</code> , progress JSON, plots, prediction sidecars

14 Closing Notes

The current design keeps the main contracts narrow:

- `ProfileInput` describes a profile.
- `FeatureBatch` describes the derived per-level representation.
- `Result` describes the model output.
- synthetic training examples provide labels without changing the runtime inference contract.

That separation is what makes the codebase easy to extend. You can change the readers, the synthetic corruption model, or the neural network independently as long as those three contracts stay stable.